*Universidad Internacional del Ecuador (UIDE)*

***Programación Orientada a objetos***

**Title:** *Fase 1*

**Professor:** María Del Carmen Salazar


**Team Members**

Ariel Yumbillo

Andrés Pesantez

Israel Farfan


Repository: https://github.com/zlkd4rk/Integrated-Project

12 octubre 2026

## 1. Problema y necesidad de la empresa

En la actualidad, la organización enfrenta serias deficiencias en el flujo de comunicación interna debido al uso de canales informales (como aplicaciones de mensajería instantánea no corporativas) y cuentas de correo personales. Esta práctica descentralizada genera los siguientes problemas críticos:

- Falta de centralización y trazabilidad: Imposibilidad de mantener un historial auditable de los mensajes enviados y recibidos formalmente entre departamentos.
- Vulnerabilidad en el control de acceso: Ausencia de mecanismos para verificar la autenticidad del remitente y garantizar la confidencialidad de la información interna.
- Ineficiencia operativa: Dificultad para localizar información histórica relevante y falta de métricas sobre los tiempos de respuesta y atención de mensajes.

Por lo tanto, surge la necesidad apremiante de implementar un sistema institucional de correo interno que garantice la persistencia, seguridad, organización y trazabilidad de las comunicaciones de la empresa.

## 2. Sistema Seleccionado

Se ha seleccionado el desarrollo de un Sistema de Gestión de Correos Internos.

El sistema permitirá que los empleados autenticados mediante credenciales seguras gestionen el ciclo de vida completo de la mensajería institucional: envío de correos, consulta de bandeja de entrada, búsqueda y filtrado de mensajes, y marcado de lectura.

- Para el alcance de esta primera fase (Fase 1), el proyecto se enfocará en las operaciones esenciales del dominio core:
- Funcionalidades de la versión inicial (MVP): Autenticación de usuario, redacción y envío de mensajes, gestión de bandeja de entrada, lectura y estado de correo (leído/no leído).
- Funcionalidades para etapas posteriores: Soporte para archivos adjuntos, recuperación de credenciales, notificaciones en tiempo real y carpetas personalizadas.

## 3. Justificación: problema, arquitectura y estadística

La elección del sistema se fundamenta en tres pilares clave:

**A. Resolución del problema de negocio**

Centraliza de manera formal la comunicación corporativa, elimina el uso de canales no autorizados y provee una bitácora fiable de interacción entre empleados.

**B. Operaciones Cliente-Servidor reales**

El sistema no se limitará a una simulación local en memoria. Se implementará una arquitectura distribuida cliente-servidor en Go utilizando una API REST sobre el protocolo HTTP/JSON:

- Cliente Go: Manejará la interfaz con el usuario y enviará peticiones HTTP (GET, POST) encapsulando los datos.
- Servidor Go: Procesará las peticiones HTTP, autenticará tokens de sesión, aplicará las reglas de negocio, validará la integridad de los datos y se comunicará con un repositorio persistente.
- Evaluación técnica real: Esta separación permitirá medir latencias de red, evaluar códigos de estado HTTP (200 OK, 400 Bad Request, 401 Unauthorized), verificar persistencia real en base de datos y manejar errores de concurrencia y validación.

**C. Generación de datos medibles para la materia de Estadística**

El sistema registrará eventos continuos que generarán un conjunto de datos (dataset) relevante para análisis estadístico descriptivo e inferencial:

- Variables cuantitativas continuas y discretas:
  - Volumen diario/semanal de mensajes: Cantidad total de correos enviados por departamento.
  - Tiempo de respuesta (latencia de lectura): Intervalo de tiempo transcurrido entre la recepción del mensaje y su marcado como leído.
  - Carga por usuario: Promedio, mediana y desviación estándar de correos recibidos y enviados por usuario.
- Variables cualitativas / categóricas:

- Estado del mensaje: Proporción de correos leídos vs. no leídos (distribuciones de frecuencia).
- Picos de tráfico: Distribución de envíos por franjas horarias durante la jornada laboral.
- Tasa de error: Porcentaje de envíos fallidos por errores de validación o autenticación.
- Todos estos indicadores se calcularán respetando la privacidad de la información, analizando exclusivamente la metadata de las transacciones sin exponer el contenido de los mensajes.



## 4. Actores y usuarios iniciales

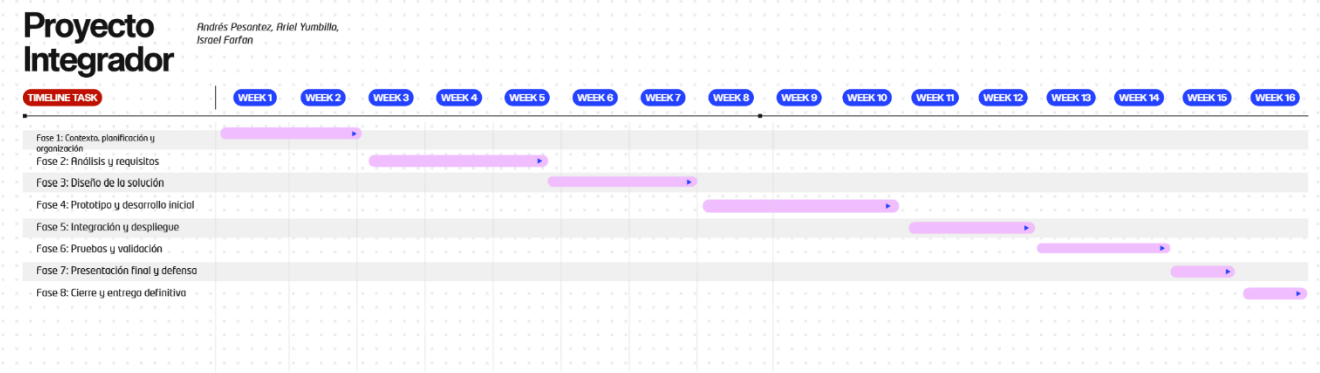
| Actor/usuario | Participación inicial |
|---|---|
| Empleado | Inicia sesión, envía correos internos, revisa su bandeja y marca mensajes como leídos. |
| Administrador | Gestiona cuentas de usuario y revisa métricas generales; no debe leer contenido privado sin autorización. |
| Cliente Go | Interfaz de usuario que consume la API del servidor. |
| Servidor Go | Autentica/valida solicitudes, aplica reglas y expone endpoints. |
| Repositorio/base de datos | Guarda usuarios, mensajes, fechas y estados de lectura. |
| Equipo de desarrollo | Analiza, diseña, implementa, prueba, documenta y presenta el proyecto. |

## 5. Cronograma de trabajo del equipo

| Semana / etapa | Actividad | Resultado |
|---|---|---|
| 1 | Acordar alcance, actores, requisitos y roles. | Documento de contexto y matriz de responsabilidades. |
| 2 | Definir contrato de comunicación y organizar el repositorio/tablero. | README inicial, licencia preliminar y tareas creadas (a cargo del equipo). |
| 3 | Diseñar clases, interfaz, relaciones y flujo cliente-servidor. | Diagrama UML inicial y revisión del diseño. |
| 4 | Validar el documento, preparar pruebas y grabar presentación. | Documentos finales; video y enlace gestionados por el equipo. |

**Imagen 1**

**Diagrama de Gantt de la distribución de las semanas basado en el documento proporcionado a los estudiantes sobre el desarrollo del Proyecto Integrador**




**Interpretación y seguimiento del cronograma:**

- **Distribución del esfuerzo:** El plan de trabajo se encuentra estructurado en 8 fases progresivas que abarcan desde la conceptualización y el análisis inicial (Fases 1 y 2) hasta el diseño de la arquitectura, desarrollo del prototipo cliente-servidor en Go, integración, pruebas exhaustivas y defensa final (Fases 3 a 8).
- **Iteración y control:** Cada fase cuenta con entregables específicos y hitos de revisión a cargo del equipo, permitiendo realizar ajustes tempranos ante posibles bloqueos técnicos detectados durante las pruebas de integración de la API y el repositorio.
- **Seguimiento de tareas:** Las actividades detalladas en este cronograma macro se desglosarán en tareas atómicas gestionadas a través del tablero de seguimiento en el repositorio de GitHub, permitiendo monitorear el avance semanal de cada integrante según los roles establecidos en la matriz de responsabilidades

## 6. Integridad académica, licencia y uso de IA

Declaración de integridad académica: el equipo declara que el trabajo se elaborará con honestidad, que identificará las fuentes utilizadas, que no presentará trabajo ajeno como propio y que todos los integrantes contribuirán de forma verificable. El contenido generado o adaptado será revisado por el equipo antes de entregarse. La declaración no sustituye las normas específicas de la institución.

Licencia preliminar: se propone la licencia MIT para el código del proyecto, sujeta a la aprobación del equipo y a las reglas de la asignatura. Los documentos académicos y materiales de terceros conservarán sus respectivas condiciones y atribuciones. Antes de publicar el repositorio, el equipo confirmará que tiene permiso para compartir todos los materiales incluidos.

Uso de herramientas de inteligencia artificial: se podrán utilizar herramientas de IA como apoyo para lluvia de ideas, organización, revisión de redacción y propuestas iniciales de diseño. Los integrantes verificarán la exactitud técnica, adaptarán el material, comprenderán lo entregado y cumplirán las reglas del docente sobre declaración de uso. La IA no reemplaza la participación del equipo ni se utilizará para inventar fuentes, resultados de pruebas o contribuciones

.

## 7. Matriz de actores, roles y responsabilidades

Sustituir los nombres genéricos por los nombres reales. Si hay menos integrantes, se pueden combinar roles.

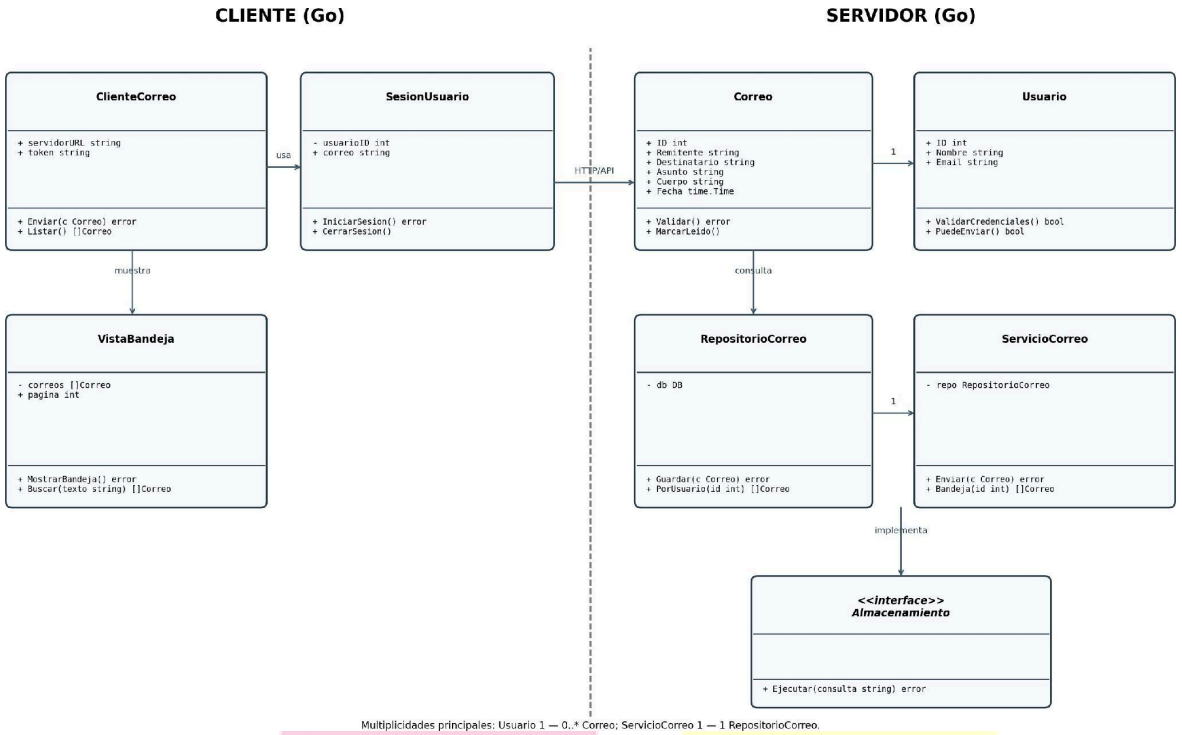
| Integrante/rol | Responsabilidades |
|---|---|
| Israel — Coordinación y análisis | Organiza reuniones, mantiene alcance y cronograma, redacta requisitos y verifica entregables. |
| Ariel — Cliente Go | Diseña las funciones de inicio de sesión, bandeja, búsqueda y envío desde el cliente. |
| Andres — Servidor/API Go | Define endpoints, validaciones, respuestas HTTP y reglas de negocio. |
| ISRAEL — Datos y pruebas | Diseña el almacenamiento, prepara datos de prueba y verifica persistencia, errores y métricas. |
| Todo el equipo | Revisar cambios, mantener README y tablero, participar en decisiones y respetar integridad académica. |

## 8. Contrato de comunicación del equipo

| Tema | Acuerdo propuesto |
|---|---|
| Canales | Chat grupal para coordinación rápida; videollamada para reuniones; repositorio Git para código/documentación; tablero de tareas para asignaciones y seguimiento. |
| Tiempos de respuesta | Responder mensajes ordinarios en un máximo de 24 horas. Si una tarea bloquea al equipo, avisar en cuanto se detecte y procurar responder en 2–4 horas dentro del horario acordado. |
| Reuniones | Una reunión breve semanal y otra cuando haya un bloqueo importante. Compartir antes los avances y después registrar acuerdos y responsables. |
| Decisiones | Buscar consenso. Si no se alcanza, votar entre los integrantes presentes; registrar decisión, motivo y tareas afectadas en el tablero o README. |
| Conflictos | Conversar respetuosamente sobre hechos y criterios, escuchar a cada parte, proponer alternativas y acordar una solución. Si persiste, solicitar mediación del docente. |
| Cambios y entregas | Crear tareas antes de trabajar, usar ramas o commits descriptivos, revisar cambios importantes y avisar cuando una tarea esté terminada. |
| Disponibilidad | Cada integrante comunica con anticipación sus horarios y posibles retrasos; no se presume disponibilidad permanente. |

## 9. Diagramas de clases UML inicial

Imagen 2
Diagrama UML de la estructura que se planteó para el proyecto.




El diagrama representa la estructura estática del sistema de gestión de correos internos, definiendo un total de 7 estructuras (clases) y 1 interfaz distribuidas en una arquitectura distribuida cliente-servidor.

**A. Separación Arquitectónica (Cliente - Servidor)**

La línea punteada central establece la frontera de comunicación entre los dos componentes del sistema:

- Lado del Cliente (Go): Comprende las estructuras ClienteCorreo, SesionUsuario y VistaBandeja. Se encargan de la interacción con el usuario, la gestión del token de autenticación y la visualización de la bandeja de entrada.
- Canal de Comunicación: La conexión entre cliente y servidor se realiza mediante solicitudes HTTP/API (REST), intercambiando datos en formato JSON.
- Lado del Servidor (Go): Contiene el núcleo de la aplicación con Correo, Usuario, ServicioCorreo y RepositorioCorreo. Maneja las reglas de negocio, la validación de credenciales y el acceso a datos.

**B. Elementos del Dominio y Encapsulamiento**

- Visibilidad y Modificadores: Se aplica el estándar UML utilizando + para miembros públicos (exportables en Go, con inicial en mayúscula) y - para miembros privados (no exportables fuera del paquete, con inicial en minúscula).
- Modelado en Go: Cada entidad se traducirá a un struct en Go junto con sus respectivos métodos asociados.

**C. Relaciones y Multiplicidades**

- Usuario — Correo $(1 \rightarrow 0..*)$: Un usuario puede ser remitente/destinatario de cero o múltiples correos dentro del sistema.
- ServicioCorreo — RepositorioCorreo $(1 \rightarrow 1)$: La capa de servicio se compone de una instancia de repositorio para ejecutar las consultas de persistencia (Guardar, PorUsuario).
- ClienteCorreo — SesionUsuario / VistaBandeja: El cliente utiliza la sesión activa para autenticar las peticiones y muestra la interfaz de la bandeja de entrada al usuario.

**D. Abstracción y Persistencia (Interfaz)**

Se define la interfaz Almacenamiento, la cual expone el método Ejecutar(consulta string) error. Esta interfaz actúa como un contrato que desacopla la lógica de negocio de la implementación concreta de la base de datos, garantizando la flexibilidad del sistema y facilitando la creación de pruebas unitarias mediante mocks. Dado que Go no utiliza herencia, este desacoplamiento y la reutilización de lógica se resuelven mediante composición e implementación implícita de interfaces.

## 10. Notas de implementación

Para el desarrollo del sistema de gestión de correos internos en Go, se adoptarán las siguientes decisiones de diseño técnico:

- **Estructuras de datos (Structs):** Cada entidad del dominio (Usuario, Correo, ClienteCorreo, etc.) se modelará mediante structs que agruparán sus propiedades y campos correspondientes. El comportamiento y la lógica de negocio se implementarán a través de métodos asociados a estos punteros a estructuras.
- **Composición en lugar de Herencia:** Dado que Go no soporta herencia clásica por clases, la reutilización de código y la extensión de funcionalidades se resolverán estrictamente mediante **composición**. Por ejemplo, un componente de servicio contendrá referencias a estructuras de repositorio u otros componentes en lugar de derivar de ellos.
- **Inyección de dependencias mediante Interfaces:** La interfaz Almacenamiento definirá el contrato que debe cumplir cualquier capa de persistencia. Esto permite desacoplar la lógica de negocio (ServicioCorreo) de la base de datos concreta, facilitando la realización de pruebas unitarias (mocking) y permitiendo cambiar el motor de almacenamiento sin alterar las reglas del servidor.
- **Separación Cliente-Servidor:** El cliente interactuará con el servidor exclusivamente a través de una API basada en HTTP, serializando y deserializando los datos en formato JSON para el intercambio de peticiones y respuestas.